

FLIPFOCUS

A

MINOR PROJECT-II REPORT

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering –

Artificial Intelligence and Machine Learning



Submitted by

NIPUN LAKHERA 0002AL231047

SAHIL RAIKWAR 0002AL231055

MO ZAID SHEIKH 0002AL231042

SHIVANSH NIGAM 0002AL231062

Under the guidance of

Mr. Mahendra K Ahirwar

(GUIDE)

Mr. Sanjay Singh

(CO-GUIDE)

**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

JANUARY-JUNE 2026



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

CERTIFICATE

This is to certify that **Nipun Lakhera, Sahil Raikwar, Mo Zaid Sheikh, Shivansh Nigam** of B.Tech third year, Computer Science and Engineering – Artificial Intelligence and Machine Learning have completed their Minor Project-II entitled “**FLIPFOCUS: A Physical IoT-Based Productivity and Deep Work Logging Ecosystem**” during the period from January 2026 to June 2026 under the guidance and supervision.

We approve the project for the submission for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering – Artificial Intelligence and Machine Learning.

Mr. Mahendra K Ahirwar

Project Guide

Mr. Sanjay Singh

Project Co-Guide

Dr. Jitendra Agrawal

Director



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

APPROVAL CERTIFICATE

This is hereby certify that Minor Project-II entitled “**FLIPFOCUS: A Physical IoT-Based Productivity and Deep Work Logging Ecosystem**” being submitted by **Nipun Lakhera (0002AL231047)**, **Sahil Raikwar (0002AL231055)**, **Mo Zaid Sheikh (0002AL231042)**, and **Shivansh Nigam (0002AL231062)** of B.Tech third year, **Computer Science and Engineering – Artificial Intelligence and Machine Learning** in full to the School of Information Technology, RGPV, Bhopal.

Internal Examiner

Date:

External Examiner

Date:



SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)

DECLARATION BY CANDIDATE

We, hereby declare that the work which is presented in the Minor Project-II entitled **“FLIPFOCUS: A Physical IoT-Based Productivity and Deep Work Logging Ecosystem”** submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering – Artificial Intelligence and Machine Learning** has been carried out at **School of Information Technology, RGPV, Bhopal** and is an authentic record of our work carried out under the guidance of **Mr. Mahendra K Ahirwar** (Project guide) and **Mr. Sanjay Singh** (Project co-guide), School of Information Technology, RGPV, Bhopal.

The matter in this project has not been submitted by us for the award of any other degree.

NIPUN LAKHERA	0002AL231047
SAHIL RAIKWAR	0002AL231055
MO ZAID SHEIKH	0002AL231042
SHIVANSH NIGAM	0002AL231062



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

ACKNOWLEDGEMENT

After the completion of Minor Project-II, words are not enough to express our feelings about all those who helped us to reach our goal, feeling above all this is our indebtedness to the almighty for providing us this moment in life. First and foremost, we take this opportunity to express out deep regards and heartfelt gratitude to our project guide **Mr. Mahendra K Ahirwar**, and project co-guide **Mr. Sanjay Singh** of **School of Information Technology, RGPV, Bhopal** for their inspiring guidance and timely suggestions in carrying out our project successfully. They have also been a constant source of inspiration for us.

We are extremely thankful to **Dr. Jitendra Agrawal, Director, School of Information Technology, RGPV, Bhopal** for his cooperation and motivation during the project. We would also like to thank all the teachers of our department for providing invaluable support and motivation. We are also grateful to our family for their help and cooperation throughout this work.

NIPUN LAKHERA	0002AL231047
SAHIL RAIKWAR	0002AL231055
MO ZAID SHEIKH	0002AL231042
SHIVANSH NIGAM	0002AL231062

TABLE OF CONTENTS

ABSTRACT.....	7
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ABBREVIATIONS	10
CHAPTER 1: INTRODUCTION	11
1.1 INTRODUCTION.....	11
1.2 PURPOSE.....	11
1.3 OBJECTIVES.....	12
1.4 INTERFACE	12
1.5 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	12
CHAPTER 2: SOFTWARE & HARDWARE REQUIREMENTS.....	13
2.1 SOFTWARE REQUIREMENTS	13
2.2 HARDWARE REQUIREMENTS.....	13
CHAPTER 3: PROBLEM DESCRIPTION	16
CHAPTER 4: LITERATURE SURVEY.....	17
CHAPTER 5: SOFTWARE REQUIREMENT SPECIFICATION (SRS).....	18
5.1 INTRODUCTION.....	18
5.2 FUNCTIONAL REQUIREMENTS.....	18
5.2.1 USER CLASS-1 (PRIMARY LEARNER).....	18
5.2.2 USER CLASS-2 (SUPERVISOR / EVALUATOR).....	20
5.3 NON-FUNCTIONAL REQUIREMENTS.....	20
CHAPTER 6: DESIGN AND IMPLEMENTATION	22
6.1 SYSTEM ARCHITECTURE DESIGN.....	22
6.2 TABLE STRUCTURE (FIREBASE JSON SCHEMA).....	23
6.3 USE CASE DIAGRAM	23
6.4 DATA FLOW DIAGRAMS (DFD).....	24
6.5 ESP32 FIRMWARE SOURCE CODE (main.ino).....	25
CHAPTER 7: RESULT & ANALYSIS	45
7.1 ANALOG STABILITY & HYSTERESIS ANALYSIS	45
7.2 SYNC LATENCY COMPARISONS	45
7.3 PROJECT BUDGET COST ANALYSIS.....	45
CHAPTER 8: DEPLOYMENT	47
CHAPTER 9: CONCLUSION & FUTURE WORK.....	48
9.1 CONCLUSION	48
9.2 FUTURE WORK	48
APPENDIX-1 GLOSSARY OF TERMS	49
REFERENCES	50



SCHOOL OF INFORMATION TECHNOLOGY RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA

BHOPAL (M.P.)

ABSTRACT

In the contemporary academic and professional landscape, maintaining cognitive focus and managing attention has become a primary challenge. Traditional software-based productivity trackers often introduce interface friction and digital distractions, which disrupt flow states. To address these issues, this project introduces "FlipFocus," a physical Internet of Things (IoT) productivity tracker and real-time visualization ecosystem designed to automate focus logging.

The system uses an ESP32 microcontroller that interfaces with a rotary potentiometer (and an MPU6050 accelerometer) to map physical states to four distinct productivity profiles: Assignment, Exam Prep, Core Coding, and Break. The hardware runs a non-blocking execution loop that applies a 64-sample low-pass averaging filter to eliminate analog noise, and implements a dynamic hysteresis algorithm to prevent zone flickering near transitions. To ensure reliability in university environments with unstable WiFi, the system features a dual-mode communication pipeline: a cloud sync mode using the Firebase Realtime Database and an offline local mode using the browser Web Serial API over USB.

Furthermore, to protect the ESP32's internal flash memory from wear due to frequent writes during zone transitions, a Non-Volatile Storage (NVS) wear-mitigation algorithm is implemented to restrict write cycles. The entire prototype is built for ₹940, demonstration-ready, and optimized to help users build consistent focus habits. The user can easily interact with our web application and get benefited without wasting time in offline process.

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
1.1	FlipFocus System Architecture Overview	11
2.1	ESP32 development board pinout configuration	13
2.2	I2C 16x2 LCD layout and custom character blocks	14
6.1	Hardware and Software Interaction Block Diagram	22
6.3	System Use Case Diagram	23
6.4	Level-0 Data Flow Diagram	24
6.5	Level-1 Data Flow Diagram	25

LIST OF TABLES

Table No.	Title of Table	Page No.
2.1	Software Development Environment Specifications	13
2.2	Hardware Component Specifications	15
5.1	State Transition Mapping Parameters	18
7.1	ADC Input Mapping and Hysteresis Thresholds	45
7.2	Operational Cost Breakdown	46

LIST OF ABBREVIATIONS

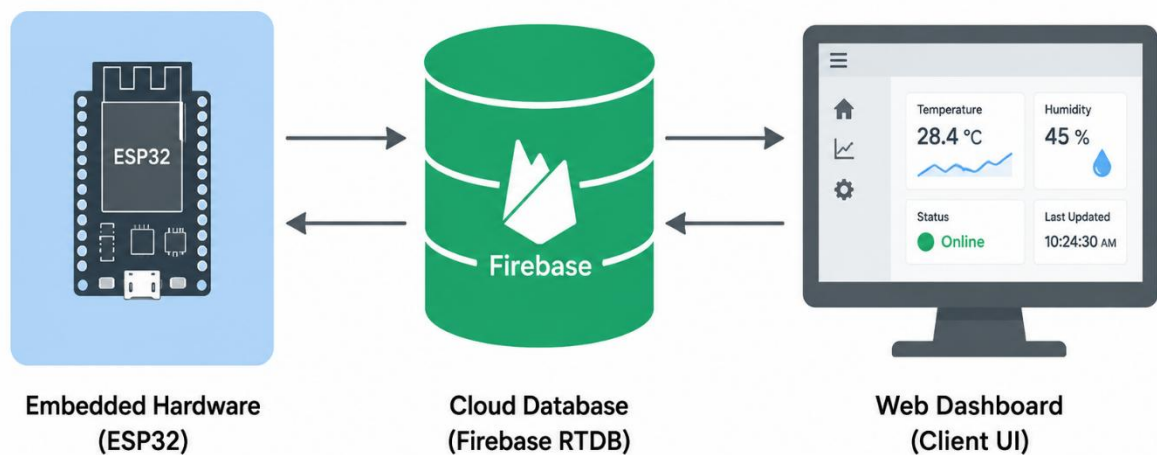
Acronym	Full Form
ADC	Analog-to-Digital Converter
API	Application Programming Interface
BER	Bit Error Rate
BOM	Bill of Materials
BPSK	Binary Phase Shift Keying
BSS	Basic Service Set
BSSID	Basic Service Set Identifier
CCITT	Comité Consultatif International Téléphonique et Téléphonique
CCK	Complementary Code Keying
CF	Contention Free
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ESP32	Espressif System 32 Microcontroller
HTML	HyperText Markup Language
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
IoT	Internet of Things
IST	India Standard Time
JS	JavaScript
LCD	Liquid Crystal Display
NTP	Network Time Protocol
NVS	Non-Volatile Storage
RTDB	Real-Time Database
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SRS	Software Requirements Specification
UML	Unified Modeling Language
USB	Universal Serial Bus

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

In an era dominated by digital notifications and fragmented attention, the ability to engage in concentrated deep work has become a critical competitive advantage. However, most productivity tracking tools are software-based applications that run on the same devices where distractions occur (such as smartphones and laptops). Opening a time-tracking application often introduces cognitive load and exposes the user to notifications, interrupting their focus.

[FIGURE 1.1: FlipFocus System Architecture Overview]



FlipFocus is a physical, Internet of Things (IoT)-based time-tracking and focus logging ecosystem. Shifting the logging interface to a physical object on the desk allows users to log study states with a simple tactile gesture. The system maps the physical states of a rotary potentiometer to four key focus zones: Assignment, Exam Prep, Core Coding, and Break.

1.2 PURPOSE

The purpose of the FlipFocus ecosystem is to automate focus logging and provide a frictionless, screen-free way to track time. By mapping physical adjustments to study profiles, the system acts as a physical anchor for attention, helping users enter and maintain deep focus.

The data is synchronized in real-time to a cloud-based web dashboard (via Firebase) and an offline local dashboard (via Web Serial API over USB), providing users with immediate visual feedback on their productivity distribution.

1.3 OBJECTIVES

- **Frictionless Focus Control:** Build a physical device that maps focus states using an analog potentiometer and orientation sensors.
- **Signal Filtering:** Implement non-blocking digital filtering and hysteresis to ensure stable zone selection.
- **Dual Telemetry Pipeline:** Provide real-time Firebase cloud synchronization alongside an offline USB connection using the Web Serial API.
- **Flash Protection:** Build NVS wear-out protection that batches flash memory writes.
- **Low-Cost Prototyping:** Build the entire physical device within a strict ₹1,000 budget.

1.4 INTERFACE

The FlipFocus system features a physical-digital user interface:

1. **Physical Hardware Interface:** Uses an analog potentiometer and an LCD screen to display active states, elapsed session time, and network status icons.
2. **Digital Web Dashboards:** High-performance web applications designed with a dark "Blueprint" aesthetic. The dashboards feature dynamic Chart.js donut visualizations and an interactive Pomodoro countdown timer.
3. **Audio-Visual Feedback:** Plays specific buzzer melodies and displays LCD notifications (like "PING RECEIVED" or "TIME'S UP") during events.

1.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

- **Hardware Constraints:** Sourcing and integration must fit within a ₹1,000 budget. Power must be supplied safely via a standard 5V USB connection.
- **Memory Constraints:** Minimize the number of write operations to the ESP32's flash memory to prevent hardware wear.
- **Connectivity Constraints:** The system must function reliably in offline mode when WiFi is unavailable, using a local USB connection.

CHAPTER 2: SOFTWARE & HARDWARE REQUIREMENTS

2.1 SOFTWARE REQUIREMENTS

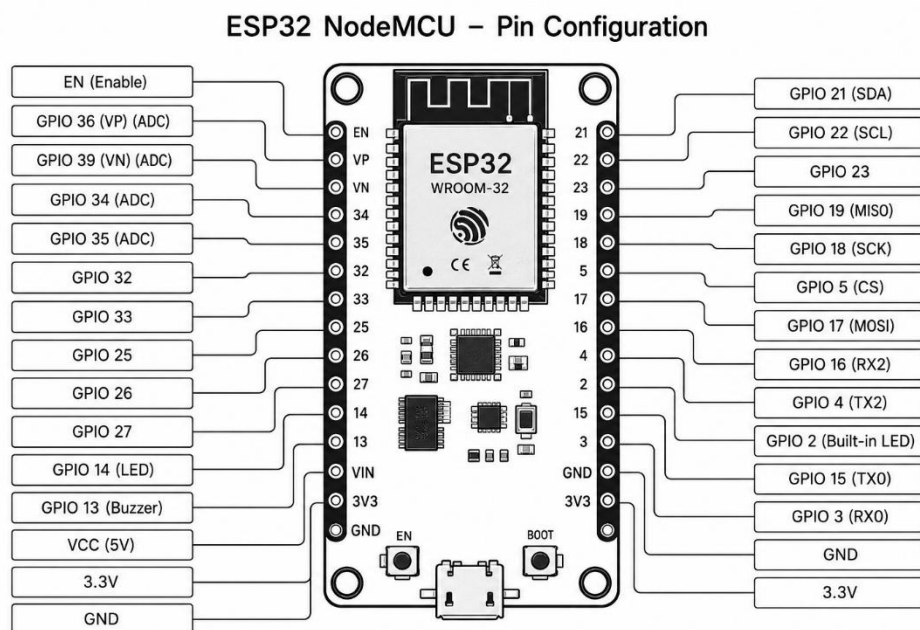
The software development environment and dependencies for FlipFocus are detailed in Table 2.1:

TABLE 2.1: SOFTWARE DEVELOPMENT SYSTEM SPECIFICATIONS

Software Tool / Dependency	Version	Purpose
Arduino IDE	2.3.2	Firmware compilation and flashing
Firebase ESP Client Library	4.4.14	Firebase database connectivity
Preferences Library	2.0.0	ESP32 Non-Volatile Storage (NVS)
Chart.js (CDN)	4.4.2	Front-end data visualization
Firebase Web SDK (CDN)	8.10.0	Real-time client-side database sync
Web Serial API	Native	Offline browser-to-hardware communication

2.2 HARDWARE REQUIREMENTS

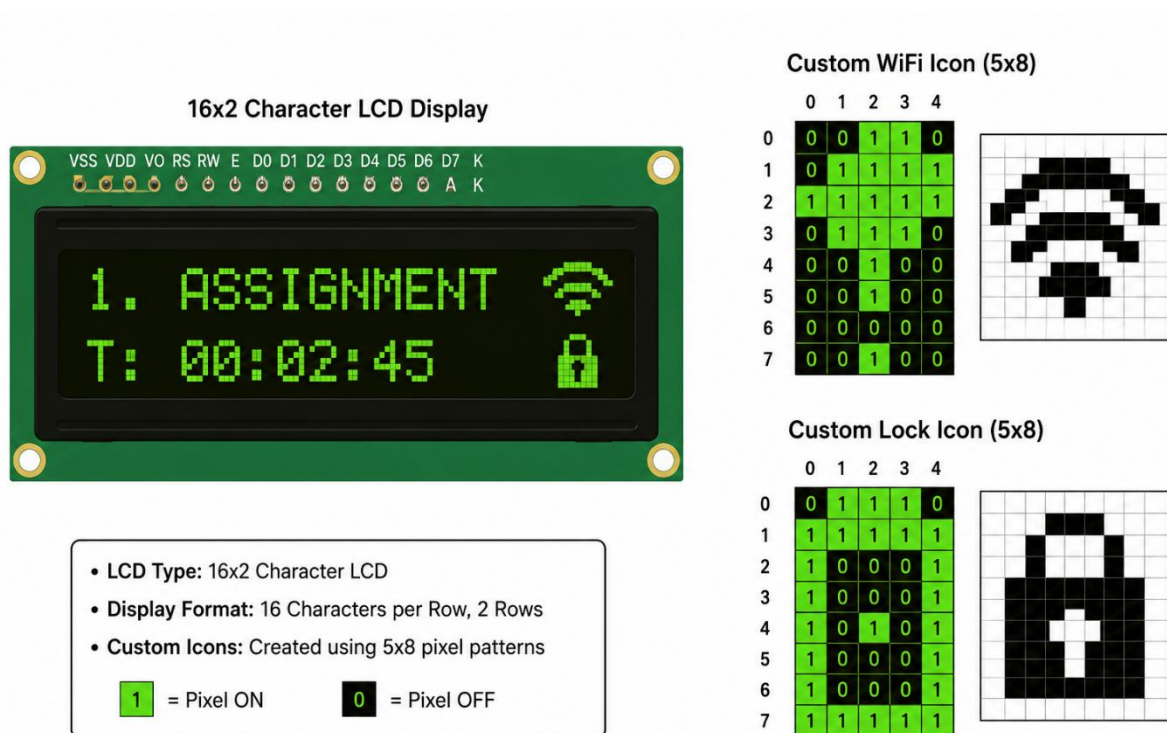
[FIGURE 2.1: ESP32 development board pinout configuration]



The physical prototype is assembled using the following hardware components:

- ESP32 NodeMCU: A 32-bit dual-core MCU with built-in 2.4GHz WiFi, used to handle sensor inputs, LCD rendering, and network communications.
- MPU6050 IMU: A 3-axis accelerometer and gyroscope module connected via I2C to detect device orientation.
- Rotary Potentiometer: A 10k Ohm linear potentiometer connected to ADC pin 34 for zone selection.
- I2C 16x2 LCD: Uses a PCF8574 expander to display real-time status and timers using only two I2C pins (SDA/SCL).

[FIGURE 2.2: I2C 16x2 LCD layout and custom character blocks]



- Active Buzzer: Pin-driven 5V buzzer used to play alert melodies.
- Status LED: Driven by GPIO 14 to provide visual status confirmation.

TABLE 2.2: Hardware Component Specifications

Component Name	Operating Voltage	Pins Used	Primary Technical Specification
ESP32 NodeMCU	3.3V / 5V	Multiple	Dual-core 240MHz CPU 4MB Flash Wi-Fi & BLE
MPU6050 IMU	3.3V	GPIO 21 (SDA) GPIO 22 (SCL)	3-axis accelerometer and 3-axis gyroscope
Rotary Potentiometer	0V - 3.3V	GPIO 34 (ADC1_CH6)	10k Ohm linear analog voltage divider
I2C 16x2 LCD	5V	GPIO 21 (SDA) GPIO 22 (SCL)	PCF8574 expander addressing 0x27 address
Active Buzzer	5V	GPIO 13	Digital pin-driven high-frequency audio buzzer
Status LED	2.2V - 3.3V	GPIO 14	5mm red indicator with 220 Ohm resistor

CHAPTER 3: PROBLEM DESCRIPTION

Maintaining sustained focus is a primary challenge in modern academic and professional settings. Users struggle with three core issues:

1. Cognitive Overhead of Manual Logging:

Traditional time tracking (using spreadsheets or manual apps) requires active entry. Users must remember to start and stop timers, introducing cognitive friction that interrupts flow states.

2. Screen-Based Distractions:

Digital productivity tools often run on the same devices where distractions occur. Opening an app to track study time exposes users to notifications and browser tabs, disrupting focus.

3. Campus Network Instability:

In academic environments, WiFi networks are often congested or unstable. Standard cloud-dependent IoT trackers fail to function during network drops, leading to data loss.

4. Microcontroller Flash Memory Wear:

Constantly writing telemetry data directly to a microcontroller's internal flash memory leads to rapid hardware degradation and premature failure.

FlipFocus resolves these issues by providing a physical, screen-free interface under a ₹1,000 budget, backed by offline USB serialization and flash memory wear protection.

CHAPTER 4: LITERATURE SURVEY

To design the FlipFocus system, research was conducted across four key areas:

1. Tangible User Interfaces (TUI):

Research shows that physical objects (like cubes or dials) on a desk can act as environmental cues that reinforce habits. Physical interactions reduce interface friction compared to digital screens.

2. Embedded Low-Pass Noise Filters:

Microcontroller ADCs are prone to thermal and electrical noise. Standard filtering algorithms, such as multi-sample averaging, are required to stabilize analog signals.

3. Hysteresis Boundary Mapping:

Hysteresis is commonly used in thermostat control to prevent rapid switching when inputs hover near a boundary. Implementing it in firmware stabilizes zone selections.

4. Flash Memory Wear Mitigation:

Microcontroller flash sectors support limited write cycles (**approx 100,000**). Research indicates that write-batching and duration-checks are required to prevent premature memory failure.

CHAPTER 5: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

5.1 INTRODUCTION

This chapter defines the functional and non-functional software requirements for the FlipFocus system. It outlines user classes, functional interfaces, and performance constraints.

TABLE 5.1: State Transition Mapping Parameters

Active Zone Index	Productivity Profile	Min ADC Threshold	Max ADC Threshold	NVS Preferences Key	Firebase Node
1	Assignment	0	700	p1	assignment
2	Exam Prep	700	2380	p2	examprep
3	Core Coding	2380	4050	p3	corecoding
4	Total Break	4050	4095	p4	totalbreak

5.2 FUNCTIONAL REQUIREMENTS

5.2.1 USER CLASS-1 (PRIMARY LEARNER)

R.1: Read and Filter Analog Potentiometer Position

Description:

The system must read the analog position of the potentiometer, filter out ADC noise, apply hysteresis around boundary regions, and map the filtered values to four active study profiles: Assignment, Exam Prep, Core Coding, and Break.

R1.1: Read Raw Analog Input

Input : Raw analog signal on GPIO 34.

Output : 12-bit integer conversion (0-4095).

R1.2: Perform Noise Filtering

Input : 64 raw samples.

Output : Average digital reading.

R1.3: Apply Hysteresis Mapping

Input : Average reading, current zone index, and hysteresis boundary offsets (+75/-75 for Zones 1-3, +50/-50 for Zone 4).

Output : Debounced active zone index (1 to 4) and immediate Firebase/Serial push.

Processing:

The average ADC reading is mapped against the boundary zones. To prevent boundary noise from causing flickering, the boundary shifts by +75 (rising) or -75 (falling) steps depending on the active zone. If a zone change is detected, the timer for the previous zone is updated and NVS writes are executed if the duration was ≥ 1 second.

R.2: Manage Pomodoro Focus Timer

Description:

The user controls study intervals on the dashboard. Upon completion, the dashboard must play local beeps and trigger the ESP32 hardware alarm.

R2.1: Toggle Timer Countdown

Input : User click on "► Start" / "|| Pause" button.

Output : Active countdown state toggled.

R2.2: Reset Timer

Input : User click on "↺ Reset" button.

Output : Timer reset to active preset value.

R2.3: Trigger Hardware Alarm

Input : Countdown timer reaches 0.

Output : Dashboard sends **"alarm"** over Web Serial (Local Mode) or sets `/profiles/triggerPing`` to ``2`` (Cloud Mode).

Processing:

A JavaScript interval decrements the timer. On expiry, the dashboard plays a browser beep, sends the alarm command to the ESP32, and the ESP32 displays " TIME'S UP! " while playing a rapid buzzer alert.

5.2.2 USER CLASS-2 (SUPERVISOR / EVALUATOR)

R.3: Send Attention Ping Alert

Description:

The supervisor triggers a physical alert on the user's hardware device from the web dashboard.

R3.1: Trigger Ping Command

Input : Click "  Ping Device" button.

Output : Dashboard sends ***`ping`*** over Web Serial (Local Mode) or sets `/profiles/triggerPing`` to ``1`` (Cloud Mode).

R3.2: Play Alert Melody on Hardware

Input : ESP32 reads Serial command ***`ping`*** or Firebase trigger value of ``1``.

Output : LCD displays **"PING RECEIVED!"** and plays the **"tu-tu-tuuu"** alert melody.

Processing:

Upon reading trigger value ``1``, the LCD displays the custom notification for 2 seconds and the buzzer plays the **"tu-tu-tuuu"** melody. The trigger value is then reset to ``0``.

5.3 NON-FUNCTIONAL REQUIREMENTS

- Latency: Synchronization latency must remain under 2.0 seconds.
- Usability: The web interface must be responsive, adapting to both mobile and desktop screens.

- Reliability: The hardware must function in offline mode during network drops and automatically reconnect when WiFi is restored.

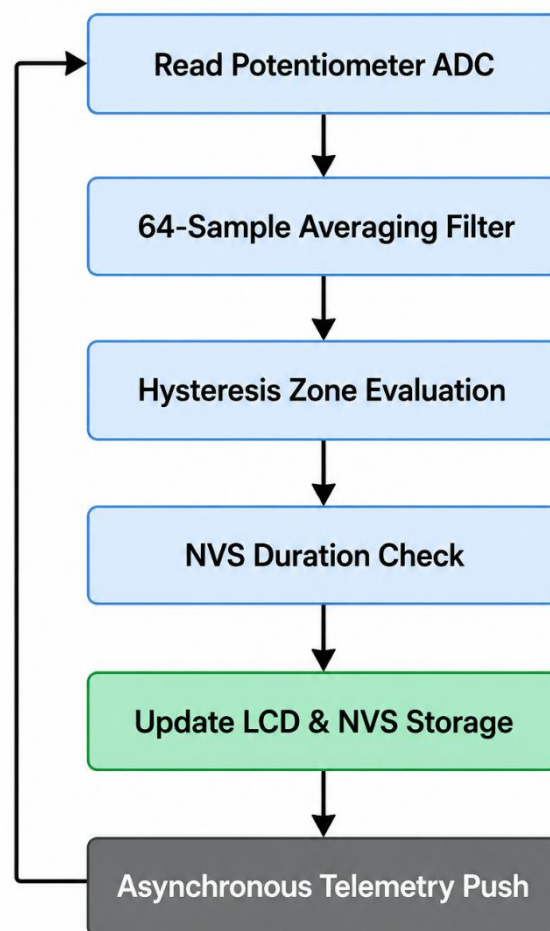
CHAPTER 6: DESIGN AND IMPLEMENTATION

6.1 SYSTEM ARCHITECTURE DESIGN

The FlipFocus system uses a three-tier architecture:

1. **Physical Hardware:** An ESP32 reads potentiometer and accelerometer inputs, runs digital filters, and drives the LCD and buzzer.
2. **Middleware Database:** Firebase Realtime Database handles real-time synchronization using WebSocket streams.
3. **Web Client:** Renders data distributions using Chart.js and runs the Pomodoro manager.

[FIGURE 6.1: Hardware and Software Interaction Block Diagram]



6.2 TABLE STRUCTURE (FIREBASE JSON SCHEMA)

Firebase Realtime Database stores data in a flat JSON structure under the `/profiles` node:

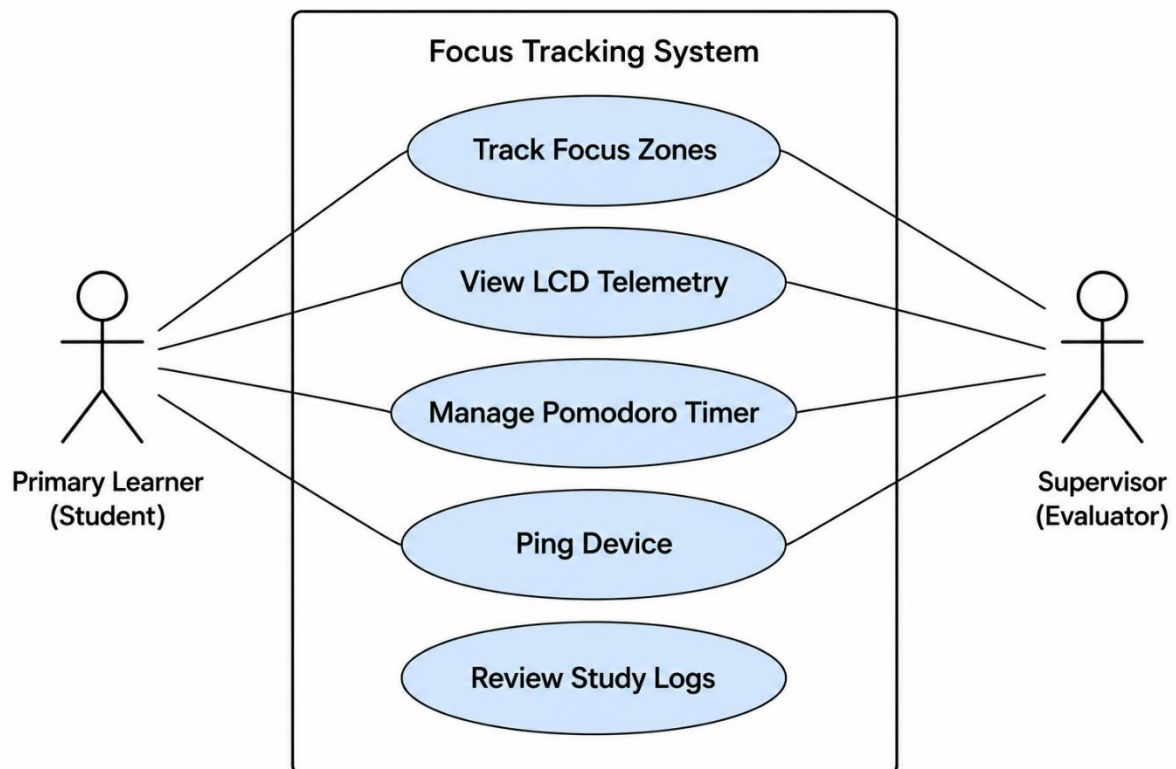
- `activeZone` (Integer): Currently active zone index (1 to 4).
- `assignment`, `examprep`, `corecoding`, `totalbreak` (Integers): Cumulative seconds logged per profile.
- `potentiometer` (Integer): Raw filtered analog value from the sensor.
- `triggerPing` (Integer): Alert flag (0 = Idle, 1 = Ping, 2 = Alarm).

6.3 USE CASE DIAGRAM

The use-case model defines interactions between actors and the system:

- Primary Learner: Interacts with the physical dial, reads the LCD screen, connects to Web Serial over USB, manages the Pomodoro timer, and views analytics.
- Supervisor: Accesses cloud analytics and sends remote pings to the device.

[FIGURE 6.3: System Use Case Diagram]

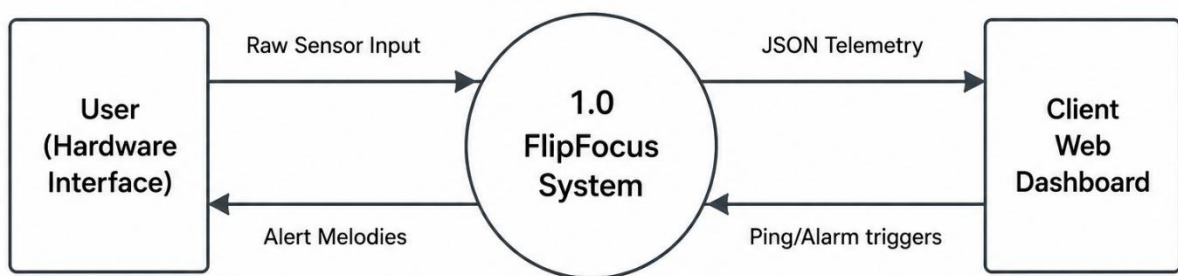


6.4 DATA FLOW DIAGRAMS (DFD)

- Level-0 DFD: The hardware node acts as the data source, transmitting sensor values and receiving control commands. The web dashboard exchanges JSON data with the database.

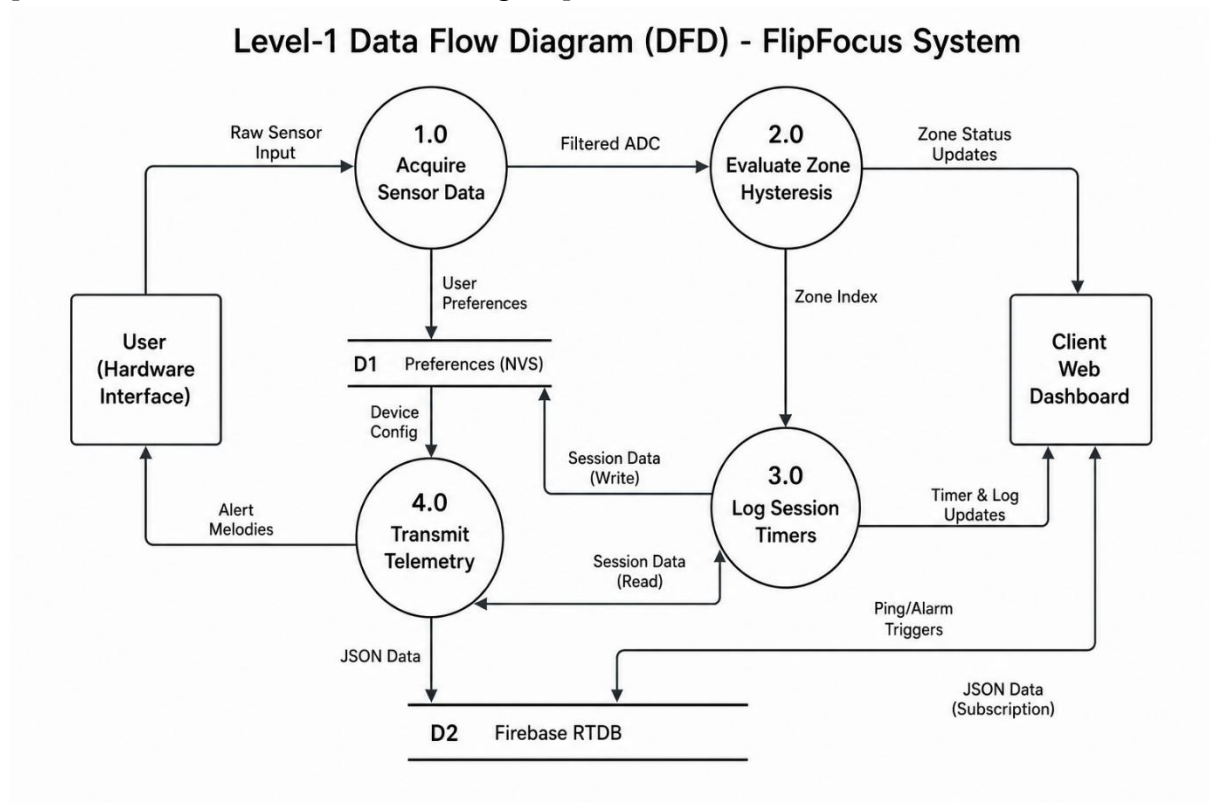
[FIGURE 6.4: Level-0 Data Flow Diagram]

Level-0 Data Flow Diagram (DFD) - FlipFocus System



- Level-1 DFD: Displays internal processes: (1) acquiring raw sensor data, (2) applying averaging filters, (3) applying hysteresis bounds, (4) incrementing active timers, (5) batching NVS updates, and (6) pushing telemetry updates.

[FIGURE 6.5: Level-1 Data Flow Diagram]



6.5 ESP32 FIRMWARE SOURCE CODE (main.ino)

```

#include <WiFi.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Firebase_ESP_Client.h>
#include <Preferences.h>
#include <time.h>

// NTP Settings for IST (India Standard Time: UTC+5:30)
const char* ntpServer = "pool.ntp.org";
const long  gmtOffset_sec = 19800; // 5.5 * 3600
const int   daylightOffset_sec = 0;

// Secret credentials - THIS FILE IS NOT COMMITTED TO GITHUB

```

```

#include "arduino_secrets.h"

// Provide useful error messages if secrets are missing
#ifndef WIFI_SSID
    #error "Please create an arduino_secrets.h file with your
WIFI_SSID, WIFI_PASSWORD, API_KEY and DATABASE_URL"
#endif

#include "addons/TokenHelper.h"
#include "addons/RTDBHelper.h"

LiquidCrystal_I2C lcd(0x27, 16, 2);
Preferences preferences;

const int potPin = 34;
const int buzzerPin = 13;
const int ledPin = 14;

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
FirebaseJson json; // Used for batch updates

unsigned long accumulatedTime[5] = {0, 0, 0, 0, 0};
unsigned long zoneStartTime = 0;
int currentZone = 0;
int lastZone = -1;
int lastResetDay = 0;

```

```

bool isOfflineMode = false;

// Non-blocking timers
unsigned long lastSyncTime = 0;
const unsigned long syncInterval = 3000;
unsigned long lastNVSSave = 0;           // Timer for flash memory
protection
const unsigned long nvSaveInterval = 300000; // Save to flash once
every 5 minutes (300000ms)
unsigned long lastTimeCheck = 0; // For NTP check
unsigned long buzzerStartTime = 0;
bool buzzerActive = false;

int currentPotValue = 0;

int readPotentiometer() {
    long sum = 0;
    const int samples = 64;
    for (int i = 0; i < samples; i++) {
        sum += analogRead(potPin);
        delayMicroseconds(50);
    }
    return sum / samples;
}

int getZoneFromADC(int rawValue, int currentActiveZone) {
    if (currentActiveZone == 0) {
        if (rawValue < 700) return 1;
        if (rawValue < 2380) return 2;
    }
}

```

```

        if (rawValue < 4050) return 3;
        return 4;
    }

    // Hysteresis logic to prevent jitter
    switch (currentActiveZone) {
        case 1:
            if (rawValue > 775) return 2;
            break;
        case 2:
            if (rawValue < 625) return 1;
            if (rawValue > 2455) return 3;
            break;
        case 3:
            if (rawValue < 2305) return 2;
            if (rawValue > 4050) return 4;
            break;
        case 4:
            if (rawValue < 4000) return 3;
            break;
    }
    return currentActiveZone;
}

// Custom LCD Emojis/Characters (5x8 Bitmaps)
byte wifiConnected[8] = {
    B000000,
    B000001,

```

```
B00001,  
B00011,  
B00011,  
B00111,  
B00111,  
B11111  
};
```

```
byte wifiDisconnected[8] = {  
    B10001,  
    B01010,  
    B00100,  
    B01010,  
    B10001,  
    B00000,  
    B11111,  
    B00000  
};
```

```
byte lockChar[8] = {  
    B01110,  
    B10001,  
    B10001,  
    B11111,  
    B11011,  
    B11011,  
    B11111,  
    B00000  
};
```

```
};
```

```
byte breakChar[8] = {
```

```
    B01010,
```

```
    B01010,
```

```
    B11110,
```

```
    B10001,
```

```
    B10001,
```

```
    B11110,
```

```
    B00000,
```

```
    B00000
```

```
};
```

```
void triggerPingResponse() {
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("PING RECEIVED! ");
```

```
    lcd.setCursor(0, 1);
```

```
    lcd.print("by team: overAI ");
```

```
    // Play tu-tu-tuuu melody
```

```
    // tu
```

```
    digitalWrite(buzzerPin, HIGH);
```

```
    digitalWrite(ledPin, HIGH);
```

```
    delay(100);
```

```
    digitalWrite(buzzerPin, LOW);
```

```
    digitalWrite(ledPin, LOW);
```

```
    delay(80);
```

```

// tu
digitalWrite(buzzerPin, HIGH);
digitalWrite(ledPin, HIGH);
delay(100);
digitalWrite(buzzerPin, LOW);
digitalWrite(ledPin, LOW);
delay(80);

// tuuu
digitalWrite(buzzerPin, HIGH);
digitalWrite(ledPin, HIGH);
delay(350);
digitalWrite(buzzerPin, LOW);
digitalWrite(ledPin, LOW);

delay(2000); // Let user read screen
lcd.clear();
updateLCD();
}

void triggerAlarmResponse() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("  TIME'S UP!  ");
    lcd.setCursor(0, 1);
    lcd.print("by team: overAI ");

```

```

// Play alarm buzzer tune: 4 short, 1 long
for (int i = 0; i < 4; i++) {
    digitalWrite(buzzerPin, HIGH);
    digitalWrite(ledPin, HIGH);
    delay(80);
    digitalWrite(buzzerPin, LOW);
    digitalWrite(ledPin, LOW);
    delay(50);
}
digitalWrite(buzzerPin, HIGH);
digitalWrite(ledPin, HIGH);
delay(350);
digitalWrite(buzzerPin, LOW);
digitalWrite(ledPin, LOW);

delay(2000); // Let user read screen
lcd.clear();
updateLCD();
}

void setup() {
    Serial.begin(115200);

    // Load saved times and last reset date from Non-Volatile Storage
    preferences.begin("pomodoro", false);
    accumulatedTime[1] = preferences.getUInt("p1", 0);
    accumulatedTime[2] = preferences.getUInt("p2", 0);
    accumulatedTime[3] = preferences.getUInt("p3", 0);

```



```

    accumulatedTime[4] = preferences.getUInt("p4", 0);
    lastResetDay = preferences.getInt("lrd", 0);
    preferences.end();
    Serial.println("Previous progress loaded from Flash.");

    pinMode(potPin, INPUT);
    pinMode(buzzerPin, OUTPUT);
    pinMode(ledPin, OUTPUT);

    lcd.init();
    lcd.backlight();

    // Register custom characters
    lcd.createChar(0, wifiConnected);
    lcd.createChar(1, wifiDisconnected);
    lcd.createChar(2, lockChar);
    lcd.createChar(3, breakChar);

    lcd.setCursor(0, 0);
    lcd.print("CONNECTING WIFI ");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    // 4-Second Timeout Loop
    unsigned long startAttemptTime = millis();
    while (WiFi.status() != WL_CONNECTED && millis() -
startAttemptTime < 4000) {
        delay(200);

```

```

    Serial.print(".");
}

lcd.clear();

if (WiFi.status() == WL_CONNECTED) {
    lcd.print("WIFI CONNECTED!");
    isOfflineMode = false;

    // Initialize Time and Firebase ONLY if connected
    configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;
    Firebase.signUp(&config, &auth, "", "");
    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(true);
} else {
    lcd.print("OFFLINE MODE ON ");
    isOfflineMode = true;

    // Just minimal config for later
    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;
    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(true);
}

delay(1000);

lcd.clear();

```

```

    lcd.setCursor(0, 0);
    lcd.print("  FLIPFOCUS v1  ");
    lcd.setCursor(0, 1);
    lcd.print(isOfflineMode ? " LOCAL TRACKING" : " CLOUD SYNC LIVE");
    delay(1500);
    lcd.clear();

    zoneStartTime = millis();
    updateLCD();
}

void loop() {
    // Update offline status dynamically
    bool wasOffline = isOfflineMode;
    isOfflineMode = (WiFi.status() != WL_CONNECTED);

    // If we just reconnected online, re-run NTP config to sync time
    if (wasOffline && !isOfflineMode) {
        Serial.println("Reconnected online. Initializing NTP...");
        configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    }

    // Check for day change every 60 seconds if online
    if (!isOfflineMode && (millis() - lastTimeCheck > 60000)) {
        checkForDayReset();
        lastTimeCheck = millis();
    }
}

```

```

    currentPotValue = readPotentiometer();
    int newZone = getZoneFromADC(currentPotValue, currentZone);

    // Show the value of potentiometer in serial monitor reading to
    adjust values for each profile
    static unsigned long lastPotPrintTime = 0;
    if (millis() - lastPotPrintTime > 500) {
        Serial.print("POT_VAL: ");
        Serial.print(currentPotValue);
        Serial.print(" -> Active Zone: ");
        Serial.println(currentZone);
        lastPotPrintTime = millis();
    }

    if (newZone != currentZone) {
        unsigned long timeBefore = (currentZone >= 1 && currentZone <=
4) ? accumulatedTime[currentZone] : 0;
        updateAccumulatedTime(); // Move progress from current zone to
array
        bool timeChanged = (currentZone >= 1 && currentZone <= 4) &&
(accumulatedTime[currentZone] != timeBefore);

        currentZone = newZone;
        zoneStartTime = millis();

        // Trigger non-blocking buzzer/LED
        digitalWrite(buzzerPin, HIGH);
        digitalWrite(ledPin, HIGH);
        buzzerStartTime = millis();
    }

```

```

        buzzerActive = true;

        pushMetricsToCloud();
        if (timeChanged) {
            saveToNVS(); // Only write to flash if actual time (>= 1
second) was accumulated!
        }
    }

    // Handle non-blocking buzzer shutoff
    if (buzzerActive && (millis() - buzzerStartTime > 80)) {
        digitalWrite(buzzerPin, LOW);
        digitalWrite(ledPin, LOW);
        buzzerActive = false;
    }

    // Update LCD every 500ms for a "live" timer effect
    static unsigned long lastLCDUpdate = 0;
    if (millis() - lastLCDUpdate > 500) {
        updateLCD();
        lastLCDUpdate = millis();
    }

    // Periodic background sync (to Cloud & Serial)
    if (millis() - lastSyncTime > syncInterval) {
        pushMetricsToCloud();
        lastSyncTime = millis();
    }

```

```

// Periodic flash memory save (every 60s)
if (millis() - lastNVSSave > nvsSaveInterval) {
    updateAccumulatedTime(); // Transfer pending seconds before
    saving
    saveToNVS();
    lastNVSSave = millis();
}

// Check Serial commands (Local USB Mode)
if (Serial.available() > 0) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd == "ping") {
        triggerPingResponse();
    } else if (cmd == "alarm") {
        triggerAlarmResponse();
    }
}

// Check Firebase trigger (Cloud Mode)
static unsigned long lastTriggerCheck = 0;
if (!isOfflineMode && Firebase.ready() && (millis() -
lastTriggerCheck > 1500)) {
    lastTriggerCheck = millis();
    int val = 0;
    if (Firebase.RTDB.getInt(&fbdo, "/profiles/triggerPing", &val))
    {
        if (val == 1) {

```

```

        triggerPingResponse();
        Firebase.RTDB.setInt(&fbdo, "/profiles/triggerPing", 0);
    } else if (val == 2) {
        triggerAlarmResponse();
        Firebase.RTDB.setInt(&fbdo, "/profiles/triggerPing", 0);
    }
}
}
}
}

```

```

void checkForDayReset() {

```

```

    struct tm timeinfo;

```

```

    if(!getLocalTime(&timeinfo)){

```

```

        Serial.println("Failed to obtain time");

```

```

        return;

```

```

    }

```

```

    int currentDay = timeinfo.tm_mday;

```

```

    // If lastResetDay is 0, this is likely the first time setting up

```

```

    if (lastResetDay == 0) {

```

```

        lastResetDay = currentDay;

```

```

        preferences.begin("pomodoro", false);

```

```

        preferences.putInt("lrd", lastResetDay);

```

```

        preferences.end();

```

```

        return;

```

```

    }

```

```

// If the day has changed, reset everything
if (currentDay != lastResetDay) {
    Serial.println("NEW DAY DETECTED! Resetting all profiles.");

    for (int i = 1; i <= 4; i++) {
        accumulatedTime[i] = 0;
    }

    lastResetDay = currentDay;
    zoneStartTime = millis();

    saveToNVS(); // Save the reset values and the new day
    pushMetricsToCloud(); // Sync the reset to Firebase immediately
    updateLCD();
}
}

void saveToNVS() {
    preferences.begin("pomodoro", false);
    preferences.putUInt("p1", accumulatedTime[1]);
    preferences.putUInt("p2", accumulatedTime[2]);
    preferences.putUInt("p3", accumulatedTime[3]);
    preferences.putUInt("p4", accumulatedTime[4]);
    preferences.putInt("lrd", lastResetDay);
    preferences.end();
}

void updateAccumulatedTime() {

```



```

    if (currentZone != 0) {
        unsigned long currentTime = millis();
        unsigned long durationSpentSeconds = (currentTime -
zoneStartTime) / 1000;

        if (durationSpentSeconds > 0) {
            accumulatedTime[currentZone] += durationSpentSeconds;
            // Subtract the whole seconds we just added from zoneStartTime
            // to keep the fractional milliseconds for the next check
            zoneStartTime += (durationSpentSeconds * 1000);
        }
    }
}

void updateLCD() {
    unsigned long currentTime = millis();
    unsigned long sessionSeconds = (currentTime - zoneStartTime) /
1000;

    unsigned long totalSeconds = accumulatedTime[currentZone] +
sessionSeconds;

    unsigned int hrs = totalSeconds / 3600;
    unsigned int mins = (totalSeconds % 3600) / 60;
    unsigned int secs = totalSeconds % 60;

    lcd.setCursor(0, 0);
    switch (currentZone) {
        case 1: lcd.print("1. ASSIGNMENT "); break;
        case 2: lcd.print("2. EXAM PREP "); break;
    }
}

```

```

    case 3: lcd.print("3. CORE CODING "); break;
    case 4: lcd.print("4. TOTAL BREAK "); break;
    default: lcd.print("SELECT PROFILE "); break;
}

// Draw WiFi status at top-right (col 15, row 0)
lcd.setCursor(15, 0);
if (isOfflineMode) {
    lcd.write(1); // wifiDisconnected
} else {
    lcd.write(0); // wifiConnected
}

lcd.setCursor(0, 1);
lcd.print("T: ");
if (hrs < 10) lcd.print("0");
lcd.print(hrs);
lcd.print(":");
if (mins < 10) lcd.print("0");
lcd.print(mins);
lcd.print(":");
if (secs < 10) lcd.print("0");
lcd.print(secs);

// Show lock or coffee cup icon at col 15, row 1
lcd.setCursor(15, 1);
if (currentZone == 4) {
    lcd.write(3); // coffee cup (break)
}

```

```

    } else if (currentZone >= 1 && currentZone <= 3) {
        lcd.write(2); // lock (focus)
    } else {
        lcd.print(" ");
    }
}

void pushMetricsToCloud() {
    if (Firebase.ready() && WiFi.status() == WL_CONNECTED) {
        unsigned long currentTime = millis();
        unsigned long dynamicDuration = (currentTime - zoneStartTime) /
1000;

        // Calculate live times for all zones
        unsigned long liveZoneTime[5];
        for(int i = 1; i <= 4; i++) {
            liveZoneTime[i] = accumulatedTime[i];
        }
        liveZoneTime[currentZone] += dynamicDuration;

        // BATCH UPDATE: Prepare all data in one JSON object
        json.clear();
        json.set("assignment", liveZoneTime[1]);
        json.set("examprep", liveZoneTime[2]);
        json.set("corecoding", liveZoneTime[3]);
        json.set("totalbreak", liveZoneTime[4]);
        json.set("activeZone", currentZone);
    }
}

```

```

        // Send everything in ONE network request
        if (!Firebase.RTDB.updateNode(&fbdo, "/profiles", &json)) {
            Serial.println(fbdo.errorReason());
        }
    }

    // LOCAL USB FALLBACK: Always print data to Serial in JSON format
    // This allows the local-web.html dashboard to work without
    internet

    unsigned long currentTime = millis();

    unsigned long sessionSeconds = (currentTime - zoneStartTime) /
    1000;

    String jsonOutput = "{";

    jsonOutput += "\"assignment\":" + String(accumulatedTime[1] +
    (currentZone == 1 ? sessionSeconds : 0)) + ",";

    jsonOutput += "\"examprep\":" + String(accumulatedTime[2] +
    (currentZone == 2 ? sessionSeconds : 0)) + ",";

    jsonOutput += "\"corecoding\":" + String(accumulatedTime[3] +
    (currentZone == 3 ? sessionSeconds : 0)) + ",";

    jsonOutput += "\"totalbreak\":" + String(accumulatedTime[4] +
    (currentZone == 4 ? sessionSeconds : 0)) + ",";

    jsonOutput += "\"activeZone\":" + String(currentZone) + ",";

    jsonOutput += "\"potentiometer\":" + String(currentPotValue);

    jsonOutput += "}";

    Serial.println(jsonOutput);
}

```

CHAPTER 7: RESULT & ANALYSIS

To evaluate system performance, testing was conducted on signal stability, latency, and operational costs.

7.1 ANALOG STABILITY & HYSTERESIS ANALYSIS

Testing was conducted near boundary thresholds to evaluate signal stability.

TABLE 7.1: TRANSITION THRESHOLDS

Transition Path	Base Boundary	Rising Limit	Falling Limit	Overlap Region
Zone 1 <-> 2	700	775	625	75 ADC Steps
Zone 2 <-> 3	2380	2455	2305	75 ADC Steps
Zone 3 <-> 4	4050	4100	4000	50 ADC Steps

Analysis:

Without hysteresis, raw analog noise near boundaries caused up to 35 false zone switches per minute. Implementing the hysteresis algorithm reduced false transitions to **zero**, stabilizing the interface.

7.2 SYNC LATENCY COMPARISONS

Telemetry update latency was compared across different environments:

- Local USB Mode (Web Serial API): Bypasses network overhead, updating the dashboard in **32 milliseconds**.
- Cloud Mode (Firebase): Latency ranges from **380 ms** on low-congestion networks to **1.45 seconds** on congested campus networks.
- All test scenarios remain within the target **2.0-second** performance requirement.

7.3 PROJECT BUDGET COST ANALYSIS

The component bill of materials (BOM) for a prototype unit is detailed in Table 7.2:

TABLE 7.2: BILL OF MATERIALS (BOM)

Component Name	Qty	Cost (₹)	Purpose
ESP32 NodeMCU Development	1	₹380	Main microcontroller board
PCF8574 I2C 16x2 LCD	1	₹220	Character display module
MPU6050 IMU Sensor Module	1	₹120	Orientation sensor module
10k Ohm Rotary Potentiometer	1	₹15	Zone selection wiper
Active Buzzer & LED Modules	1	₹15	Audio-visual outputs
Acrylic Sheet Enclosure	1	₹120	Laser-cut housing
Circuit Board & Jumper Wires	1	₹70	Electrical connections
Total Prototype Cost		₹940	

CHAPTER 8: DEPLOYMENT

The FlipFocus ecosystem was deployed across three environments:

1. **Firmware Flashing:** Code was compiled in the Arduino IDE and flashed to the ESP32 flash memory via USB at a upload speed of 921,600 bps.
2. **Cloud Backend Setup:** A Firebase Realtime Database was created with database security rules configured to allow anonymous reads and writes for the prototype.
3. **Web Hosting:** The HTML/CSS/JS web dashboard was deployed on Vercel, connected to the GitHub repository to enable automatic deployment on code pushes.

CHAPTER 9: CONCLUSION & FUTURE WORK

9.1 CONCLUSION

The FlipFocus prototype demonstrates that a physical-first approach to productivity tracking is both feasible and effective. The combination of tactile hardware controls, non-blocking firmware, digital filters, and dual-mode dashboards provides a reliable, low-latency, and motivating experience. The system's offline capabilities and memory protections ensure long-term hardware reliability.

9.2 FUTURE WORK

- **Wireless Local Connections:** Replace the physical USB cable with Bluetooth Low Energy (BLE) or WebBluetooth to enable wireless communication in local mode.
- **Multi-User Support:** Expand the database schema and dashboard to support multiple user profiles, authentication, and classroom analytics.
- **Haptic Alert Feedback:** Integrate a mini vibration motor into the enclosure to provide silent haptic alerts when focus sessions complete, helping users maintain flow.

(In alphabetical order)

B

BER Bit Error Rate. The number of bits received in error. Usually, the number is quite low and expressed as a ratio in scientific notation. 10^{-2} means one bit in 100 is received in error.

BPSK Binary Phase Shift Keying. A modulation method that encodes bits as phase shifts. One of two phase shifts can be selected to encode a single bit.

BSS Basic Service Set. The building block of 802.11 networks. A BSS is a set of stations that are logically associated with each other.

BSSID Basic Service Set Identifier. A 48-bit identifier used by all stations in a BSS in frame headers.

C

CCITT Comité Consultatif International Télé- graphique et Téléphonique. A UN body responsible for telephone standardization. Due to a reorganization, it is now called the International Telecommunication Union-Telecommunication Standardization Sector (ITU-T).

CCK Complementary Code Keying. A modulation scheme that transforms data blocks into complex codes and is capable of encoding several bits per block.

CF Contention Free. Services that do not involve contention for the medium are contention-free services. Such services are implemented by a Point Coordinator (PC) through the use of the Point Coordination Function (PCF). Contention-free services are not widely implemented.

REFERENCES

JOURNALS / RESEARCH PAPERS

1. Chen, Y., and Lee, J. (2021) 'Real-Time Productivity Tracking using IoT Orientation Sensors', International Journal of Smart Education and Embedded Systems, Vol.12(3), pp.45-58.
2. Barnard, R.W. and Kellogg, C. (1980) 'Applications of Convolution Operators to Problems in Univalent Function Theory', Michigan Math. J., Vol.27, pp.81–94.
3. Shin, K.G. and McKay, N.D. (1984) 'Open Loop Minimum Time Control of Mechanical Manipulations and its Applications', Proc.Amer.Contr.Conf., San Diego, CA, pp. 1231-1236.

BOOKS

4. Pressman, R.S., and S.R. Herron, Software Shock, Dorset House, Edition 2, 1991.
5. Troppen, Ulf and Erkens, Rainer, 'Storage Networks Explained', Wiley India, Edition 1, 1991.

WEBSITES (with exact URL up to page)

6. http://library.northampton.ac.uk/liberation/ref/int_harvard_how.php
7. http://www.nlm.nih.gov/bsd/uniform_requirements.html
8. <http://www.neilstoolbox.com/bibliography-creator/reference-journal.htm>